Portfolio Risk, Sensitivity, and Constrained Allocation

Mathematical foundations, C++ implementation, and independent validation

Eshan Garkhedkar

October 9, 2026

Abstract

This project develops an auditable numerical engine for portfolio risk, attribution, sensitivity, and contribution-only allocation. We derive the mathematical models, implement them in C++ with a scoped reverse-mode differentiation tape, and retain independent Python reference calculations. The central distinction is between arithmetic correctness, optimization correctness, and the empirical adequacy of estimated covariance. The paper states the domains of the derivatives, formulates the constrained quadratic problem, documents data assumptions, and separates measured validation from unperformed empirical work. Historical covariance is not treated as a known law of future returns. This is a research and educational software project.

Contents

1. Definitions, prerequisites, and risk estimation **2**
Practical question . . . 2
Portfolio quadratic form . . . 2
Sample estimator and normalization . . . 3
Shrinkage and its precise status . . . 3
Annualization and interpretation . . . 3

2. Differentiation and a worked example **4**
From calculus to decisions . . . 4
Risk attribution . . . 4
New cash changes the denominator . . . 5
Hand-worked example . . . 5
Sensitivity to a valid covariance path . . . 5
Implemented custom reverse mode . . . 6

3. Constrained allocation **6**
Practical question and domain . . . 6
Feasibility before optimization . . . 7
Algorithm and projection . . . 7
Optimality and independent validation . . . 8

4. Architecture and data methodology **8**
Implemented separation of responsibilities . . . 8
Provider adapter and limits . . . 9

Calendar and observation policy . . . 9
Interface and deployment boundary . . . 10

5. Numerical verification and reproducibility **10**
Evidence and reproduction . . . 10
Tolerances and interpretation . . . 11
Worked interpretation . . . 11
Remaining empirical work . . . 12
Integration validation and measured timings . . . 12
Comprehension questions . . . 13

6. References and glossary **13**
Glossary . . . 14

1. Definitions, prerequisites, and risk estimation

Practical question

How much dispersion does a fixed portfolio have under an estimated covariance model, and which assumptions enter that estimate?

For asset i, a simple one-period return is $r_{t,i} = P^*_{t,i}/P^*_{t-1,i} - 1$. Here P^* denotes a positive, consistently adjusted historical price series whose economic interpretation has been checked. This notation alone does not establish that dividends are reinvested. Current holdings require a separate unadjusted valuation price.

Let $r_t \in \mathbb{R}^n$, $w \in \mathbb{R}^n$, $w_i \geq 0$ and $\mathbf{1}^T w = 1$. Returns and weights are dimensionless. If h_i are dollar holdings, $V = \sum_i h_i > 0$ and $w_i = h_i/V$. Covariance is in squared-return units for a declared period; volatility is in return units for the same period.

Expectation is a probability-weighted average. Variance $\mathrm{Var}(X) = E[(X - E[X])^2]$ measures squared deviations around it. Covariance $\mathrm{Cov}(X, Y) = E[(X - E[X])(Y - E[Y])]$ measures joint deviations. Correlation divides covariance by the product of nonzero standard deviations. A zero-variance variable has no defined correlation coefficient under this formula.

Portfolio quadratic form

With fixed weights, $r_p = w^T r$. Expanding the square gives

$$\mathrm{Var}(r_p) = E\left[\left(\sum_i w_i(r_i - E[r_i])\right)^2\right] = \sum_i \sum_j w_i w_j \Sigma_{ij} = w^T \Sigma w.$$

Thus $\sigma(w) = \sqrt{w^T \Sigma w}$. Covariance is symmetric and positive semidefinite (PSD): $a^T \Sigma a = \mathrm{Var}(a^T r) \geq 0$ for every real a. The spectral theorem writes $\Sigma = Q \Lambda Q^T$; PSD means nonnegative eigenvalues. Eigenvalues measure variance along orthogonal directions, not individual asset risks.

Singularity means some direction has zero variance under this model. For positive-definite Σ, the spectral condition number is $\lambda_{max}/\lambda_{min}$; a large ratio warns that inversion can amplify perturbations. Risk evaluation needs matrix multiplication, not inversion. Singularity is therefore valid

and must not be rejected simply because a Cholesky factorization fails. See MOSEK's portfolio formulation.

Sample estimator and normalization

For $T \geq 2$ aligned return rows, $\bar{r} = T^{-1} \sum_t r_t$ and centered matrix $X \in \mathbb{R}^{T \times n}$:

$$S = \frac{1}{T-1} X^T X.$$

This uses the sample normalization; under independent identically distributed rows with finite covariance it is unbiased. Rows of market data need not meet those assumptions. Since $a^T S a = \|Xa\|^2/(T-1)$, this construction preserves PSD. Centering implies $\operatorname{rank}(S) \leq \min(n, T-1)$. Pairwise deletion with different date sets need not preserve PSD; the product must use a common set of complete daily returns.

Example with three return rows $(.01, .02), (-.01, 0), (0, -.02)$ has zero mean and

$$S = \begin{pmatrix} .0001 & .0001 \\ .0001 & .0004 \end{pmatrix}.$$

The reference independently matches this result and NumPy's `cov(..., ddof=1)`.

Shrinkage and its precise status

For a PSD estimate S, define $\mu = \operatorname{tr}(S)/n$ and

$$S_\alpha = (1-\alpha)S + \alpha\mu I, \qquad 0 \leq \alpha \leq 1.$$

It is PSD as a convex combination of PSD matrices. If $\alpha > 0$ and $\mu > 0$, all eigenvalues $(1-\alpha)\lambda_i + \alpha\mu$ are positive. This improves conditioning but introduces a structured bias toward equal variances and zero covariance; it is not a guarantee of better future allocation.

The engine implements a fixed-α sensitivity path. Separately, Python fits the Ledoit–Wolf scaled-identity estimator through scikit-learn and records the estimated coefficient. Its covariance-loss criterion is not realized portfolio returns. See the original paper and estimator documentation.

The fitted estimator uses the centered empirical covariance $S_{ML} = X^T X/T$, while the sample option uses $S = X^T X/(T-1)$. Tests reconstruct the fitted result as $(1-\hat{\alpha})S_{ML} + \hat{\alpha}\operatorname{tr}(S_{ML})I/n$. Both outputs are multiplied by the declared annualization factor. Thus estimator comparisons include a normalization difference as well as shrinkage. A fixed arbitrary coefficient is never labeled a fitted Ledoit–Wolf estimate.

Annualization and interpretation

For an additive K-period return sum, covariance equals $K\Sigma_d$ only if cross-period covariance terms vanish and period covariances are identical. With serial dependence the omitted cross terms matter. Annual volatility $\sqrt{252}\sigma_d$ is a convention under these assumptions, not the exact variance of compounded annual simple returns. Display both period and multiplier. The example in chapter 2 is already annual and must not be annualized again.

This is a constant-weight risk estimate. A buy-and-hold historical account has drifting weights; reproducing that path requires holdings evolution. Historical covariance estimates future risk only through a modeling assumption of relevant history. The interface compares windows of 63, 126 and 252 completed common daily returns when available. It labels their spread sensitivity rather than calibrated uncertainty.

Implementation: reference.py and estimation.py. Validation includes hand values, scaling, permutation, PSD rejection, singular cases and estimator normalization. Exchange-calendar processing is implemented and tested with synthetic dates; empirical estimation quality remains unverified.

2. Differentiation and a worked example

From calculus to decisions

For a fixed symmetric Σ, expand $(w+\delta)^T\Sigma(w+\delta)$ to obtain

$$v(w+\delta) = v(w) + 2\delta^T\Sigma w + \delta^T\Sigma\delta.$$

Therefore $\nabla v = 2\Sigma w$. Applying the scalar chain rule to $\sqrt{v}$ yields

$$g = \nabla_w \sigma = \frac{\Sigma w}{\sigma}, \qquad \sigma > 0.$$

The derivative is with respect to unconstrained coordinates even though valid portfolios lie on a simplex. A feasible first-order direction d must satisfy $\mathbf{1}^T d = 0$ and remain inside nonnegativity bounds. A transfer from A to B has $d = e_B - e_A$, so

$$\sigma(w+\epsilon d) - \sigma(w) = \epsilon(g_B - g_A) + O(\epsilon^2).$$

At a boundary a feasible direction may allow only one sign of ϵ. Do not confuse this derivative with increasing B while holding all other weights fixed.

Risk attribution

Volatility is homogeneous of degree one for positive scalings. Euler's identity gives $w^T g = \sigma$; directly,

$$\sum_i RC_i = \sum_i w_i \frac{(\Sigma w)_i}{\sigma} = \frac{w^T\Sigma w}{\sigma} = \sigma.$$

RC_i has the same units as volatility. Capital share w_i and risk share RC_i/σ answer different questions. Contributions can be negative, even in a long-only portfolio, if covariance gives a sufficiently strong hedge; they are an attribution, not independent stand-alone risks. The attribution framework is described in MOSEK's risk budgeting chapter.

New cash changes the denominator

For dollar holdings h, $H = \sum_i h_i > 0$, and $w = h/H$:

$$\frac{\partial w_i}{\partial h_j} = \frac{\delta_{ij} - w_i}{H}, \qquad \frac{\partial \sigma}{\partial h_j} = \frac{g_j - w^T g}{H} = \frac{g_j - \sigma}{H}.$$

The units are volatility per dollar. This derivative includes the effect on all weights. Adding dollars proportionally to every existing holding leaves weights unchanged: $h^T \nabla_h \sigma = 0$. A transfer of b dollars keeps total wealth constant and instead has derivative $b(g_B - g_A)/H$.

For a fixed contribution B fully distributed through purchases x, the feasible total is constant: $w(x) = (h + x)/(V + B)$. Its objective gradient in purchase coordinates is $2\Sigma(h + x)/(V + B)^2$. Off the budget constraint, that fixed-denominator extension is a computational definition, not the normalization for arbitrary additional wealth. Both give the same directional derivative along feasible transfers of purchases.

Hand-worked example

Take $w = (.6, .4)$ and annual $\Sigma = \begin{pmatrix} .04 & .006 \\ .006 & .01 \end{pmatrix}$. The asset volatilities are .2 and .1; the implied correlation is $.006/(.2 \cdot .1) = .3$.

$$\Sigma w = (.0264, .0076), \quad v = .6^2(.04) + 2(.6)(.4)(.006) + .4^2(.01) = .01888.$$

Thus $\sigma = 0.1374045123$, $g = (.1921334282, .0553111384)$, and $RC = (.1152800569, .0221244554)$. A accounts for 83.8983% of total volatility attribution despite holding 60% of capital.

A transfer of one percentage point of portfolio weight from A to B has linear change $-.0013682229$ and exact change $-.0013611369$ in volatility units. Multiply by 100 to express volatility percentage points: approximately $-.13682$ versus $-.13611$.

Start with $h = (600, 400)$ dollars and buy 100 of B using new cash. Weights become $(600/1100, 500/1100)$. Exact volatility change is $-.0072425026$ versus the local dollar approximation $-.0082093374$. A 10% wealth increase is large enough that the approximation error is visible. The conclusion concerns percentage-return volatility; dollar risk can increase when invested wealth grows.

Sensitivity to a valid covariance path

Keep base S, target $F = \mu I$, and weights fixed. On $S_\alpha = (1 - \alpha)S + \alpha F$:

$$\frac{\partial \sigma}{\partial \alpha} = \frac{w^T(F - S)w}{2\sigma}.$$

Only $\alpha \in [0, 1]$ is admitted. At endpoints use admissible one-sided perturbations for checks. This derivative does not differentiate the estimated Ledoit–Wolf coefficient with respect to data. Changing a single correlation arbitrarily can destroy PSD; use model-valid paths instead.

Implemented custom reverse mode

The C++ tape stores immutable node values and parent edges with local derivatives. Each reverse pass allocates fresh zero adjoints, initializes the output adjoint to one, and traverses backward to add chain-rule contributions. For $z = uv$, accumulate $\bar{u} \mathrel{+}= \bar{z}v$ and $\bar{v} \mathrel{+}= \bar{z}u$. Addition, subtraction, multiplication, division and square root are implemented. If $u = v$, both edge contributions accumulate. For example, $q = x^2$ and $f = q + q$ produce $df/dx = 4x$, including the shared intermediate correctly.

A tape owns node storage. Handles retain shared ownership and carry a generation number to reject cross-tape mixing and use after reset. Graphs are rebuilt per calculation, including every optimizer gradient evaluation; stored local derivatives are never reused with changed inputs. Division by zero, square root at nonpositive values and nonfinite arithmetic fail explicitly. The risk wrapper returns zero volatility with its usual gradient unavailable, even when a special constant-zero model admits a derivative. With m operations and bounded parents per node, forward evaluation and one reverse pass require $O(m)$ work and storage. The dense quadratic graph has $O(n^2)$ operations; no speed advantage over the closed form is asserted.

AD evaluates derivatives of the implemented arithmetic graph up to floating-point effects; it does not cure statistical model error. The closed-form volatility gradient is simple and remains an independent oracle. AD supplies implementation depth and handles composed normalization and parameter paths. See Baydin et al., section 3.

Independent formulas remain in reference.py. The C++ engine evaluates risk, normalized-dollar sensitivities and fixed-shrinkage sensitivities through the tape. Python tests compare these against analytical formulas and finite differences; native tests also exercise handle lifetime, reset and shared nodes under address and undefined-behavior sanitizers.

3. Constrained allocation

Practical question and domain

Given nonnegative dollar holdings $h \in \mathbb{R}^n$, wealth $V = \mathbf{1}^T h$, contribution $B \geq 0$, and $D = V + B > 0$, distribute purchases x without selling. Assume a fixed PSD covariance, continuous dollar amounts, full investment of B and optional post-contribution caps $c_i \in [0, 1]$.

$$\min_x f(x) = \frac{(h+x)^T \Sigma (h+x)}{D^2}, \qquad \mathbf{1}^T x = B, \quad 0 \leq x_i \leq u_i = c_i D - h_i.$$

The objective has squared-return units; x and u are dollars. The gradient has squared-return units per dollar:

$$\nabla f(x) = \frac{2\Sigma(h+x)}{D^2}, \qquad \nabla^2 f = \frac{2\Sigma}{D^2} \succeq 0.$$

PSD curvature proves convexity: along every line the second derivative is nonnegative. The box intersected with the budget hyperplane is convex and compact when feasible. A continuous objective has a minimizer there. Strict positivity of curvature on nonzero feasible directions ensures uniqueness; singular covariance can instead produce many optimal allocations. Weight agreement is consequently an inadequate solver check.

Feasibility before optimization

With upper bounds only, feasibility is equivalent to $u_i \geq 0$ for all i and $\sum_i u_i \geq B$. Necessity follows by summing upper bounds; sufficiency follows by filling nonnegative capacities until B is used. An existing overweight asset may become permissible after dilution by the contribution. Evaluate $h_i \leq c_i(V + B)$, not $h_i/V \leq c_i$.

For $h = (600, 400)$, $B = 200$, and $c = (.6, .6)$, $u = (120, 320)$ and the budget is feasible. Tightening the first cap to .4 makes its capacity -120, so no purchase-only solution exists. The reference implements these checks in `purchase_bounds`; the implemented solver's residual tolerances are specified below.

Algorithm and projection

Projected gradient is scoped, transparent, and independently testable. Start from a feasible point and iterate

$$x^{k+1} = \Pi_C(x^k - \eta \nabla f(x^k)), \quad C = \{x : 0 \leq x \leq u,\ \mathbf{1}^T x = B\}.$$

For $L = 2\lambda_{max}(\Sigma)/D^2 > 0$, use $0 < \eta \leq 1/L$ as an initial fixed-step design. If $L = 0$, every feasible point is optimal and no reciprocal step is formed. Projected gradient is the proximal-gradient method with an indicator of the feasible set; see Parikh and Boyd, section 4.2.

To project a trial point z, minimize $\frac{1}{2}\|x - z\|^2$ over C. Stationarity with an equality multiplier τ and box clipping yields

$$x_i(\tau) = \min\{u_i, \max\{0, z_i - \tau\}\}, \quad \sum_i x_i(\tau) = B.$$

The sum is continuous and nonincreasing. For $0 < B < \sum_i u_i$, bracket τ between $\min_i(z_i - u_i)$ and $\max_i z_i$, then bisect until the budget residual meets a documented scale-aware tolerance. The projected point is unique, though the multiplier need not be. Handle $B = 0$ and $B = \sum_i u_i$ directly. Validate projection feasibility, idempotence, and the variational inequality $(z - \Pi_C z)^T(y - \Pi_C z) \leq 0$ for feasible y.

The implemented coordinates are $\ell = h/D$, $y = x/D$, $b = B/D$ and $a = c - \ell$. Thus $w = \ell + y$, with feasible set $C_y = \{y : 0 \leq y \leq a,\ \mathbf{1}^T y = b\}$. This avoids dollar-scale conditioning and remains defined when $B = 0$. For $s = \lambda_{max}(\Sigma) > 0$, define $Q = \Sigma/s$ and $F(y) = (\ell + y)^T Q(\ell + y)$. The gradient $2Q(\ell + y)$ has Lipschitz constant 2, so the code takes step $\eta = 1/2$. Each projection uses rescaled bisection for 160 iterations and a final budget-residual correction inside the box. The baseline projects equal purchase fractions onto the feasible set. Zero contribution or zero covariance has an immediate feasible optimum.

To see why this step descends, let $d = y^{k+1} - y^k$. Projection optimality gives $\nabla F(y^k)^T d \leq -\|d\|^2/\eta$. The Lipschitz-gradient bound then gives $F(y^{k+1}) \leq F(y^k) - (1/\eta - L/2)\|d\|^2$. For $L = 2$ and $\eta = 1/2$, the coefficient is one. In exact arithmetic, convex projected gradient also satisfies $F(y^k) - F(y^*) \leq \|y^0 - y^*\|^2/(2\eta k)$ for $k \geq 1$. This is an objective bound, not a guarantee of unique weights. Floating-point projection error and a finite iteration limit make explicit diagnostics necessary.

Optimality and independent validation

For the original dollar problem, KKT stationarity is

$$\nabla f + \nu \mathbf{1} - \lambda + \gamma = 0,$$

with unrestricted ν, nonnegative λ, γ, feasibility, $\lambda_i x_i = 0$ and $\gamma_i(x_i - u_i) = 0$. For this convex quadratic on a polyhedron, an appropriate first-order variational inequality characterizes global optima. A convenient numerical diagnostic is

$$G_\eta(x) = \frac{x - \Pi_C(x - \eta \nabla f(x))}{\eta}.$$

It vanishes exactly at a constrained optimum. The implementation stops when the scaled mapping norm $2\|y - \Pi_{C_y}(y - \nabla F(y)/2)\|_2$ is at most 10^{-9} by default, or after 10,000 updates. This absolute, dimensionless threshold is adjustable. Multiplying this norm by s gives the mapping norm in weight coordinates for the unscaled variance objective with step $1/(2s)$; it is not a dollar-coordinate norm. The output also reports dollar budget residual, maximum weight-bound violation, iteration count, stopping reason, objective and improvement over the feasible baseline. Constraint feasibility admits only 10^{-12} weight-scale roundoff before projection. Independent tests compare full-rank and rank-deficient objectives with OSQP, emphasizing objective agreement instead of unique weight agreement.

The C++ solver obtains the composed objective gradient through the tape by default; an analytical-gradient mode is also checked. Covariance stays fixed within each solve. The interface repeats solves across predeclared windows and covariance estimators when enough observations exist. Synthetic convergence and solver-agreement evidence is reported in chapter 5; real-market allocation stability remains unmeasured.

Minimum estimated variance ignores expected return, investor objectives, tax, fees, execution and estimation error. It answers this explicit constrained mathematical question; it does not define the best personal investment portfolio.

4. Architecture and data methodology

Implemented separation of responsibilities

The dependency chain is provider adapter, validated returns, Python covariance estimation, C++ risk/AD/optimization, then Streamlit presentation. The C++17 engine accepts arrays and explicit parameters without network or dashboard dependencies. CMake and scikit-build-core package a pybind11 extension. Eigen supplies dense arithmetic and symmetric eigendecomposition. The independent NumPy reference remains separate.

Bindings validate numerical inputs in the core and translate exceptions. Returned arrays own their Python-visible storage. The allocator releases the Python interpreter lock while computing. Result metadata includes engine version, estimation method, normalization, window, annualization, dataset hash and diagnostics. These fields describe reproducibility inputs, not immutable historical data.

Provider adapter and limits

The Alpaca adapter loads credentials server-side from environment variables or Streamlit secrets. Only market-data endpoints are called. Historical requests explicitly select IEX or SIP and adjustment mode `all`; pagination follows page tokens, rejecting repeated tokens and excessive pagination. Authentication and malformed responses fail clearly. Transport errors, HTTP 429 and server failures receive at most four attempts with bounded waits. There is no silent feed fallback. Alpaca's access documentation distinguishes coverage and subscription requirements.

The adapter has a bounded, 15-minute in-memory history cache, isolated per dashboard session and credential identity. Cache keys include symbols, range, feed and adjustment mode. Requests end at least 15 minutes before analysis time rather than in the future. On expiry or manual refresh the entire adjusted window is fetched, and snapshots are compared for added, removed and revised observations, including older dates. Returned histories are defensive copies. This audits incremental changes without mixing adjustment vintages; it does not reduce network volume. An overlap-only refresh cannot guarantee detection of arbitrary revisions outside its overlap.

Historical bars documentation lists adjustment choices, but an option name alone does not establish dividend reinvestment economics. Known split/dividend cases, account entitlement and display/redistribution permissions remain unverified. Checked-in fixtures are synthetic. Mocked HTTP tests establish software behavior, not successful live access. For an independently specified corporate event, the audit helper compares the adjusted-price return with the holder's wealth return

$$r_{\text{event}} = \frac{kP_{\text{after}} + d}{P_{\text{before}}} - 1,$$

where k is new shares per old share and d is cash per old share. A two-for-one split from 100 to 50 gives zero wealth return. A cash dividend of one with prices 100 and 99 also gives zero. Backward price adjustment is not generally identical to cash-wealth arithmetic when the ex-date price also changes for other reasons. Tests intentionally detect that discrepancy; the helper cannot certify a provider convention without checked live events.

Calendar and observation policy

Only XNYS sessions whose close plus a 15-minute publication allowance precedes the analysis time enter daily-return preparation. Explicit UTC timestamps are converted to New York session dates. Prices must be positive and finite; duplicate observations fail. Missing history at the latest completed session fails. The selected universe is USD US stocks and unleveraged ETFs; user confirmation is required, while automated asset eligibility validation remains pending.

Prices are reindexed to the expected calendar before returns are computed. Each return requires prices on adjacent expected sessions, so a missing Monday cannot turn Friday-to-Tuesday movement into a daily return. Only then are common complete return rows retained. Effective row counts, excluded dates and a SHA-256 identity of the prepared returns are recorded. No forward-filling occurs. Tests cover gaps, weekends, incomplete sessions, pagination, retries, staleness and estimator normalization.

Current shares use separately retrieved, unadjusted latest trades. A price older than 15 minutes during an open session, or older than 15 minutes before the last completed close while closed, is flagged stale. Prices more than 30 seconds into the future and nonpositive prices are rejected. Stale share valuation stops the interface. A last trade is an observation, not an executable quote; this

threshold is an explicit policy, not a provider guarantee. Dollar and hypothetical-weight inputs do not need the valuation request.

Interface and deployment boundary

The Streamlit app supports a credential-free synthetic example, attribution, exact versus local purchase effects, contribution allocation and diagnostics. Market mode adds dollar/share/weight input, feed selection and comparisons over sample and Ledoit–Wolf estimates at 63, 126 and 252 observations when available. Zero-volatility gradients and infeasible caps are explained explicitly.

Deployment configuration includes Python and Linux build dependencies, but no hosted app has been deployed or private-access tested. Ubuntu continuous integration compiles the engine and exercises the app test harness; that is distinct from a hosted session. Before deployment, confirm private-app capacity, build/import behavior and sharing settings against Streamlit's documentation. Live credentials and market-data permissions remain prerequisites for live operation.

5. Numerical verification and reproducibility

Evidence and reproduction

The initial 13 NumPy reference tests passed locally on macOS arm64 with Python 3.12.14 and NumPy 2.3.5. On October 9, 2026, all 27 tests passed on Ubuntu with both Python 3.10 and 3.12 at commit `8438207`. This includes seven C++ numerical tests, five data/estimator tests and two Streamlit interaction tests. Native sanitizer checks and both example reports also passed in workflow 37956387933.

Install a C++17 compiler, CMake and Eigen3, then run from the repository root:

```
python -m pip install '.[test,app]'
python -m unittest discover -s tests -v
python -m examples.two_asset --output work/two-asset.json
python -m examples.engine_report --output work/engine-report.json
```

The C++ report records synthetic data identity, source hashes, runtime environment, derivative errors and allocation diagnostics. The Python 3.12.15 / NumPy 2.5.3 run measured maximum weight-gradient error $2.78 \cdot 10^{-17}$ and dollar-gradient error $4.07 \cdot 10^{-20}$ against the independent reference. The worked allocation had zero reported budget, bound and projected-gradient residuals. These values describe this fixture, not worst-case error bounds. Each Python job also stores JSON artifacts. Source hashes identify the evaluated code; later documentation edits do not change the historical record.

Test family	Independent evidence
Risk and attribution	Hand variance .01888; NumPy formula; Euler identity
Gradients	Analytical, AD and central-difference comparisons
Dollar normalization	Recomputed holdings and feasible transfer derivatives
Tape semantics	Shared nodes, repeated passes, lifetime, reset and domains

Test family	Independent evidence
Projection	Budget and bounds, idempotence, variational inequality
Allocation	OSQP objectives for full-rank and singular matrices
Degenerate cases	Zero budget/covariance, iteration limit, infeasible caps
Covariance	PSD validation, rank deficiency, scaling and permutation
Estimation	Hand covariance and reconstructed fitted shrinkage
Data	Mock pagination/retries, calendar gaps, staleness and caching
Interface	Synthetic display, infeasible caps, universe confirmation

Native tests additionally run with AddressSanitizer and UndefinedBehaviorSanitizer. Passing this finite set provides evidence for covered cases, not a proof of memory safety for every input. Tests use synthetic inputs without provider credentials.

Tolerances and interpretation

Calculations use float64. Covariance validation tolerates asymmetry and negative eigenvalues only at $100n\epsilon_{mach} \max |S_{ij}|$. It symmetrizes and clips negative eigenvalues only inside that allowance, then uses the same effective matrix for risk and gradients. Materially indefinite matrices are rejected. This relative rule has no absolute floor hiding errors at small scales. The risk API reports the Frobenius norm of its covariance correction.

Weights must sum to one within 10^{-12}; they are not silently renormalized. Dollar normalization is explicit. Central finite differences have truncation error $O(h^2)$ and cancellation error roughly $O(\epsilon_{mach}/h)$. Multiple step sizes are used, with relative tolerances up to $3 \cdot 10^{-6}$ for coarse steps and absolute allowances near zero. See gradcheck mechanics; PyTorch is not a dependency.

OSQP checks request absolute and relative tolerances 10^{-10}. Comparisons allow objective relative error 10^{-7} and absolute error 10^{-10}, alongside engine convergence and feasibility diagnostics. Singular problems may have several minimizers, so equal weights are not required. Near-zero volatility can make sensitivities ill-conditioned even when variance evaluation is stable. Extreme floating-point scale testing remains a hardening task.

Worked interpretation

The synthetic portfolio has volatility .1374045123 and contributions .1152800569 and .0221244554. Fixed-alpha shrinkage changes volatility from .1374045 at zero to .1140175 at one; this spread measures model dependence, not statistical confidence. The purchase example shows increasing error in a local derivative approximation as purchases grow.

For the same covariance, holdings $(600, 400)$, contribution 200 and caps $(.6, .6)$, the solution purchases $(0, 200)$ and ends at $(.5, .5)$ with variance .0155. Along the feasible interval the variance

derivative favors additional B, but the no-sale bound prevents reducing A below .5. The optimum is at a boundary despite a nonzero unconstrained gradient.

Remaining empirical work

Unperformed work includes live corporate-action validation, entitlement checks, private hosted operation and chronological real-data analysis. The implemented research protocol fits windows of 63, 126 and 252 observations through a cutoff, then evaluates the next 21 observations, using nonoverlapping test blocks. Each cutoff poses an independent contribution problem with fixed initial holdings of 100 dollars per asset and a 200-dollar contribution. It is not a cumulative trading simulation. For fitted weights w_t, it compares predicted annual variance with $252\,\mathrm{Var}_{\mathrm{test}}(r_s^T w_t)$, using sample normalization within the test block. The calibration ratio and successive allocation L^1 changes summarize variance fit and stability. Future-perturbation tests ensure that changing test observations cannot change the first fitted allocation. Present-day adjusted history and a surviving-symbol universe can still introduce vintage or selection bias despite correct slicing.

The benchmark compares validated risk APIs against the vectorized NumPy reference, with five warm-up calls and seven batches of 30 calls at 2, 10 and 50 assets. It records median and percentile timings, compiler, platform, NumPy build configuration and thread settings. These are end-to-end costs including validation, eigendecomposition and Python/C++ crossing, not isolated kernel timings. Isolating pure binding overhead remains a separate benchmark. No investment-return result is claimed.

Integration validation and measured timings

At commit `aeea037`, all 34 tests passed on Python 3.10 and 3.12, alongside native sanitizer checks in run 37958614693. The data tests cover cache isolation, old revisions, removed observations, invalid timestamps and event arithmetic. The projection now subtracts a common offset before rescaling, preserving translation invariance for a large shared offset such as $(10^{16}, 10^{16})$ with unit bounds and unit budget; its projection is $(.5, .5)$. Tests also vary dollar units from 10^{-6} to 10^{6} and covariance scale from 10^{-12} to 10^{12}.

The synthetic factor study uses seed 24109, 504 observations and three assets. It completed 72 evaluations with mean calibration ratio 1.0096. This is a reproducible exercise of the protocol under synthetic data, not empirical evidence about market covariance or an estimated confidence level. Full evaluation records are retained in the workflow artifacts.

On the Python 3.12 Ubuntu x86-64 runner, using GCC 13.3 and one configured BLAS/OpenMP thread, measured median microseconds per validated API call were:

Assets	NumPy	C++ analytical	C++ AD
2	34.44	3.48	4.48
10	45.14	9.98	30.41
50	285.10	182.34	884.69

The analytical C++ path was faster for these cases, while the AD path was slower than NumPy at 50 assets. Graph construction and traversal have a material cost. This result depends on the tested APIs, runner and dimensions; it is not a claim that C++ or AD is universally faster. The benchmark JSON records percentile timings and the numerical-library environment.

Comprehension questions

1. Why does a 60% capital holding contribute about 84% of volatility?
2. Why is a new-dollar derivative $(g_j - \sigma)/V$ instead of g_j/V?
3. Why can singular covariance be valid while optimal weights are nonunique?
4. Why can a constrained optimum have a nonzero ordinary gradient?
5. Which assumptions justify square-root-of-time annualization?

6. References and glossary

Sources consulted on 2026-10-08. Documentation is mutable; recheck when implementing integrations. Entries distinguish material actually consulted from full proofs not yet reviewed.

1. **MOSEK, Portfolio Optimization Cookbook**, chapter 2, and chapter 9. Consulted portfolio formulation and Euler attribution. Our contribution-only allocation is a project-specific specialization.
2. **Olivier Ledoit and Michael Wolf (2004)**, "A well-conditioned estimator for large-dimensional covariance matrices," *Journal of Multivariate Analysis* 88(2), 365–411, DOI 10.1016/S0047-259X(03)00096-4. Publisher abstract and indexed author-paper introduction consulted for motivation and scope of asymptotic covariance-loss claims. Full proof review remains pending; author-hosted PDF retrieval was unsuccessful. No theorem from the unread proof is claimed as newly verified.
3. **scikit-learn**, LedoitWolf API. Consulted the scaled-identity formula and centering contract. The estimator wrapper is implemented and empirical normalization is tested.
4. **Atılım Güneş Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, Jeffrey Mark Siskind (2018)**, "Automatic Differentiation in Machine Learning: a Survey," *JMLR* 18(153), 1–43. Publisher record, paper. Consulted section 3, reverse traversal and accumulation. Its title does not imply machine learning is part of our project.
5. **Neal Parikh and Stephen Boyd (2014)**, "Proximal Algorithms," *Foundations and Trends in Optimization* 1(3), 127–239. Author-hosted paper. Consulted projected gradient as proximal gradient for a set indicator; project-specific capped-simplex derivation is provided in chapter 3.
6. **PyTorch**, Gradcheck mechanics. Consulted central finite differences and analytical comparison. Reference tests do not use PyTorch.
7. **pybind11**, official documentation. Consulted C++/Python integration overview. The extension uses CMake and scikit-build-core; tests exercise arrays and handle lifetime.
8. **Alpaca**, Market Data API overview and historical bars endpoint. Consulted feed/access distinctions, pagination and adjustment options. Data entitlement and display/redistribution terms are not yet validated.
9. **Streamlit**, Community Cloud, privacy/sharing, and dependencies. Consulted private-app allowance and Linux dependency mechanism. No hosted build has been performed.

Glossary

Term	Meaning here
Covariance	Expected product of centered returns
PSD	Every quadratic form is nonnegative
Eigenvalue	Variance scaling along an eigenvector direction
Conditioning	Sensitivity of a numerical problem to input perturbations
Shrinkage	Blend an estimate with a structured target
Gradient	Vector of unconstrained coordinate derivatives
Directional derivative	Local change along a specified direction
Adjoint	Derivative of the output with respect to an intermediate node
Tape	Stored computational graph and intermediate values
Projection	Nearest feasible point in the chosen norm
KKT conditions	Feasibility, stationarity, multiplier signs and complementarity
Residual	Quantified violation of a constraint or optimality condition
Annualization	Conversion under declared time-aggregation assumptions
Sensitivity	Change under specified perturbations, not a confidence interval